Zidane Karim -- 4/30/2026

Table of Contents

Generate L source steering vectors & s matrix

```
function [A, S, B, V] = generate_sensor_data(M, N, theta_deg, d_lambda, PdB,
PndB)

    L = length(theta_deg);

    theta_rad = deg2rad(theta_deg);

    S = zeros(M, L);

    % Column vector of spatial indices (0 to M-1)
    m_idx = (0:M-1)';

    for l = 1:L
        omega = 2 * pi * d_lambda * cos(theta_rad(l));
        S(:, l) = (1/sqrt(M)) * exp(-1i * m_idx * omega);
    end
```

2. Create the B matrix

```
var_B = 10.^(PdB / 10);

B = zeros(L, N);
for l = 1:L
    B(l, :) = sqrt(var_B(l)) * (randn(1, N) + 1i * randn(1, N)) / sqrt(2);
end
```

3. Create the V matrix

```
var_V = 10^(PndB / 10);

V = sqrt(var_V) * (randn(M, N) + 1i * randn(M, N)) / sqrt(2);

A = S * B + (1/sqrt(M)) * V; % A matrix

end
```

Part 1: Signal Generation

```
d_lambda = 1/2;
M = 100;
N = 200;
sourcePowers = [0, -2, -4];
noisePower = 10;
theta1 = [10, 25, 70];
theta2 = [10, 12, 70];

[A1, S1, B1, V1] = generate_sensor_data(M, N, theta1, d_lambda,
sourcePowers, noisePower);
[A2, S2, B2, V2] = generate_sensor_data(M, N, theta2, d_lambda,
sourcePowers, noisePower);

R1 = (1/N) * (A1 * A1');
R2 = (1/N) * (A2 * A2');
```

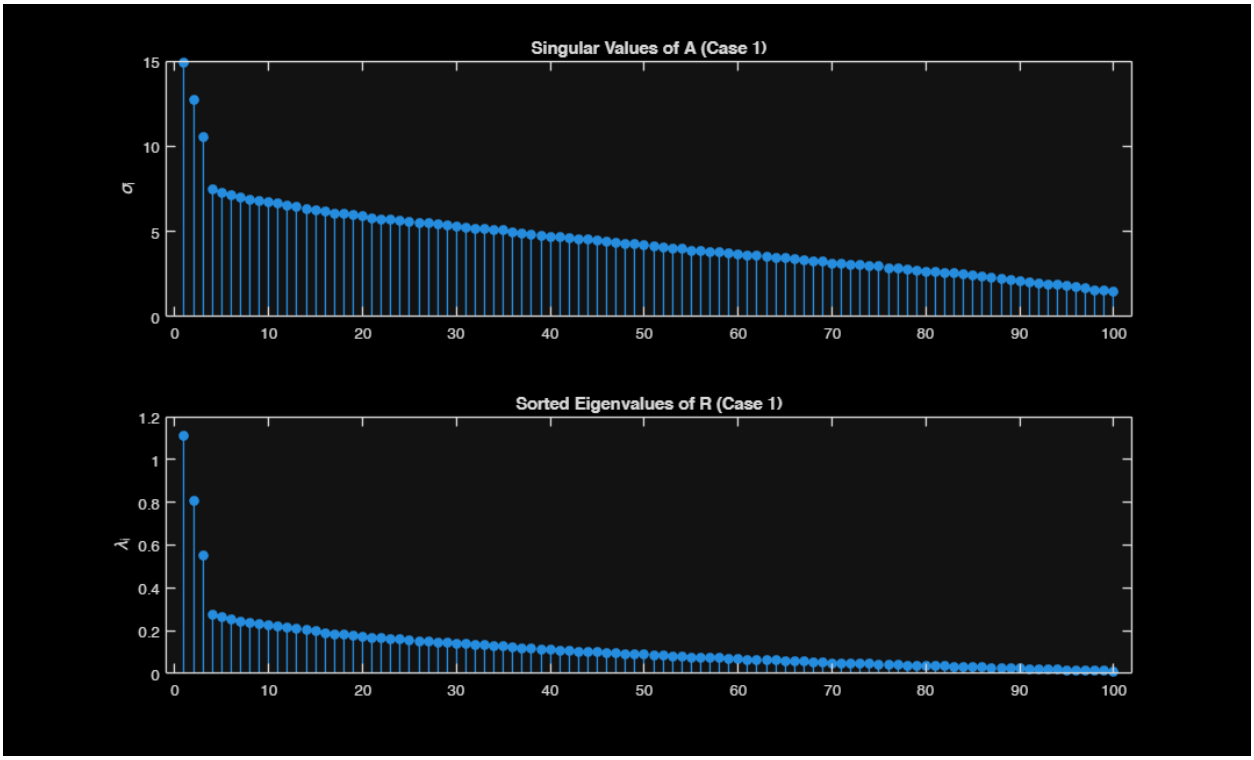
Part 2: Analysis

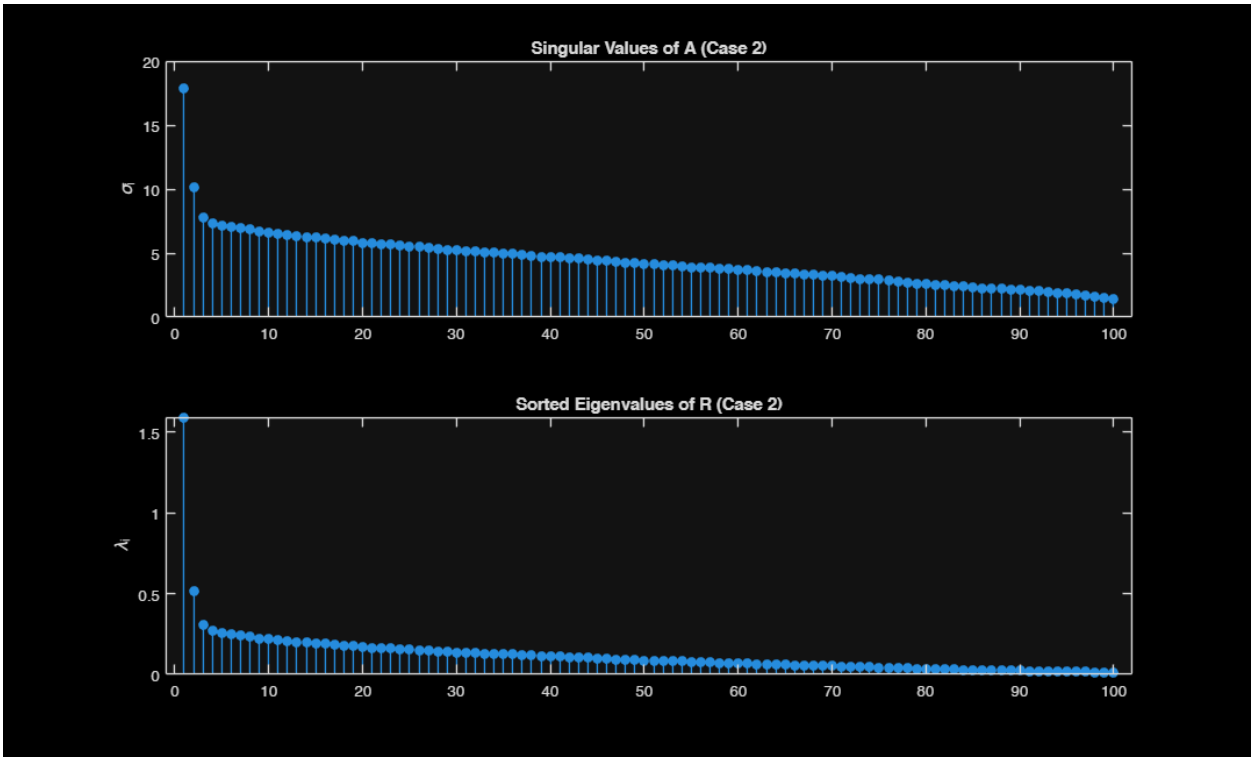
```
[U1, S_mat1, V_mat1] = svd(A1);
s_vals1 = diag(S_mat1);

[U2, S_mat2, V_mat2] = svd(A2);
s_vals2 = diag(S_mat2);

% sort
[eigvec1, eigval01] = eig(R1);
[eigval1, idx1] = sort(diag(eigval01), 'descend');
eigvec1 = eigvec1(:,idx1);

[eigvec2, eigval02] = eig(R2);
[eigval2, idx2] = sort(diag(eigval02), 'descend');
eigvec2 = eigvec2(:, idx2);


figure;
subplot(2,1,1);
stem(s_vals1, 'filled');
title('Singular Values of A (Case 1)');
ylabel('\sigma_i');

subplot(2,1,2);
stem(eigval1, 'filled');
title('Sorted Eigenvalues of R (Case 1)');
ylabel('\lambda_i');

figure;
subplot(2,1,1);
stem(s_vals2, 'filled');
title('Singular Values of A (Case 2)');
ylabel('\sigma_i');
```

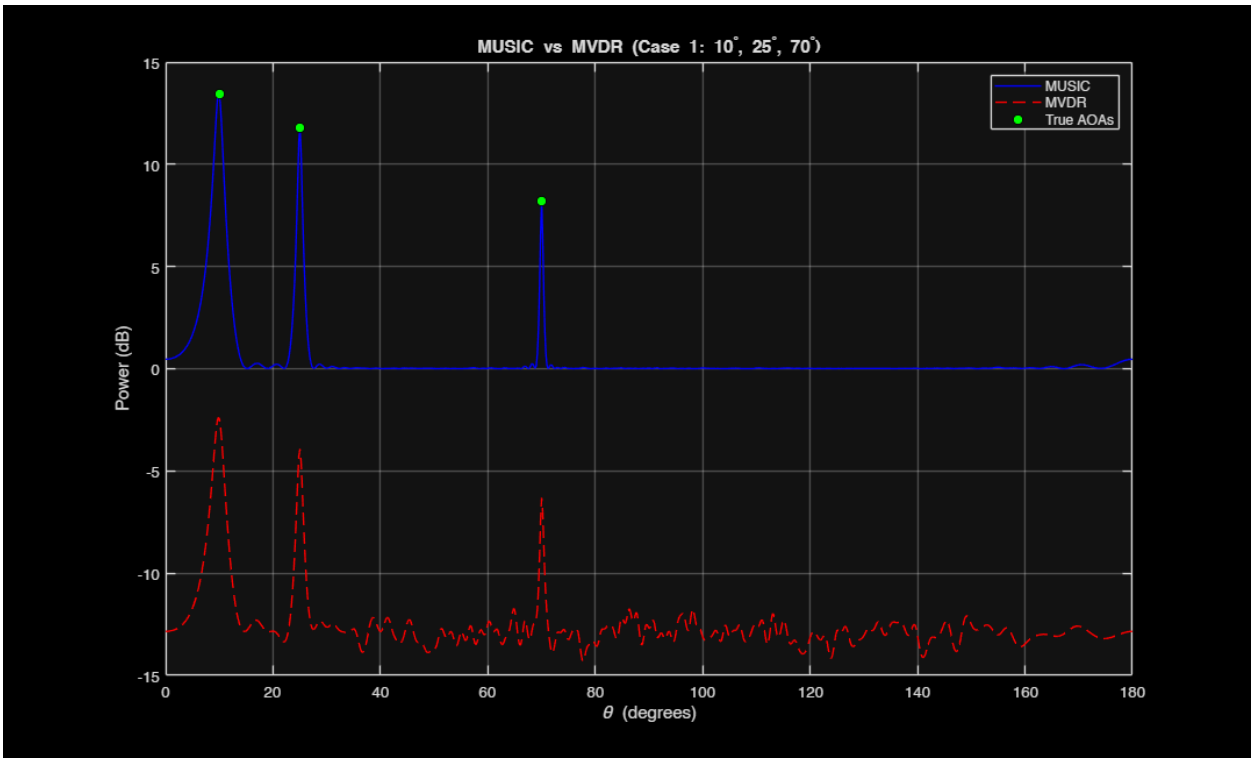
```
subplot(2,1,2);
stem(eigval2, 'filled');
title('Sorted Eigenvalues of R (Case 2)');
ylabel('\lambda_i');

% sigma3/sigma4
% Calculate the ratio of the third to fourth singular values for both cases
sigmaRatio1 = s_vals1(3) / s_vals1(4);
sigmaRatio2 = s_vals2(3) / s_vals2(4);

disp(['Sigma Ratio Case 1: ', num2str(sigmaRatio1)]);
disp(['Sigma Ratio Case 2: ', num2str(sigmaRatio2)]);

% lambda3/lambda4
% Calculate the ratio of the third to fourth eigenvalues for both cases
lambdaRatio1 = eigval1(3) / eigval1(4);
lambdaRatio2 = eigval2(3) / eigval2(4);

disp(['Lambda Ratio Case 1: ', num2str(lambdaRatio1)]);
disp(['Lambda Ratio Case 2: ', num2str(lambdaRatio2)]);

% project matrix to the noise subspace
P1 = eigvec1(:, 4:end) * eigvec1(:, 4:end)';
P2 = eigvec2(:, 4:end) * eigvec2(:, 4:end)';

A1_noise = P1 * A1; % Projected A1
A2_noise = P2 * A2; % Projected A2

Rinverse1 = inv(R1);
Rinverse2 = inv(R2);

theta_range = 0:0.2:180;
S_MUSIC1 = zeros(size(theta_range));
S_MVDR1 = zeros(size(theta_range));
% case 1
for i = 1:length(theta_range)
    omega = 2 * pi * d_lambda * cos(deg2rad(theta_range(i)));
    s_theta = (1/sqrt(M)) * exp(-1i * (0:M-1)' * omega);

    % MUSIC: 1 / (s' * P_N * s)
    S_MUSIC1(i) = 1 / abs(s_theta' * P1 * s_theta);
    % MVDR: 1 / (s' * R_inv * s)
    S_MVDR1(i) = 1 / abs(s_theta' * Rinverse1 * s_theta);
end

S_MUSIC2 = zeros(size(theta_range));
S_MVDR2 = zeros(size(theta_range));
% case 1
for i = 1:length(theta_range)
    omega = 2 * pi * d_lambda * cos(deg2rad(theta_range(i)));
    s_theta = (1/sqrt(M)) * exp(-1i * (0:M-1)' * omega);

    % MUSIC: 1 / (s' * P_N * s)
```

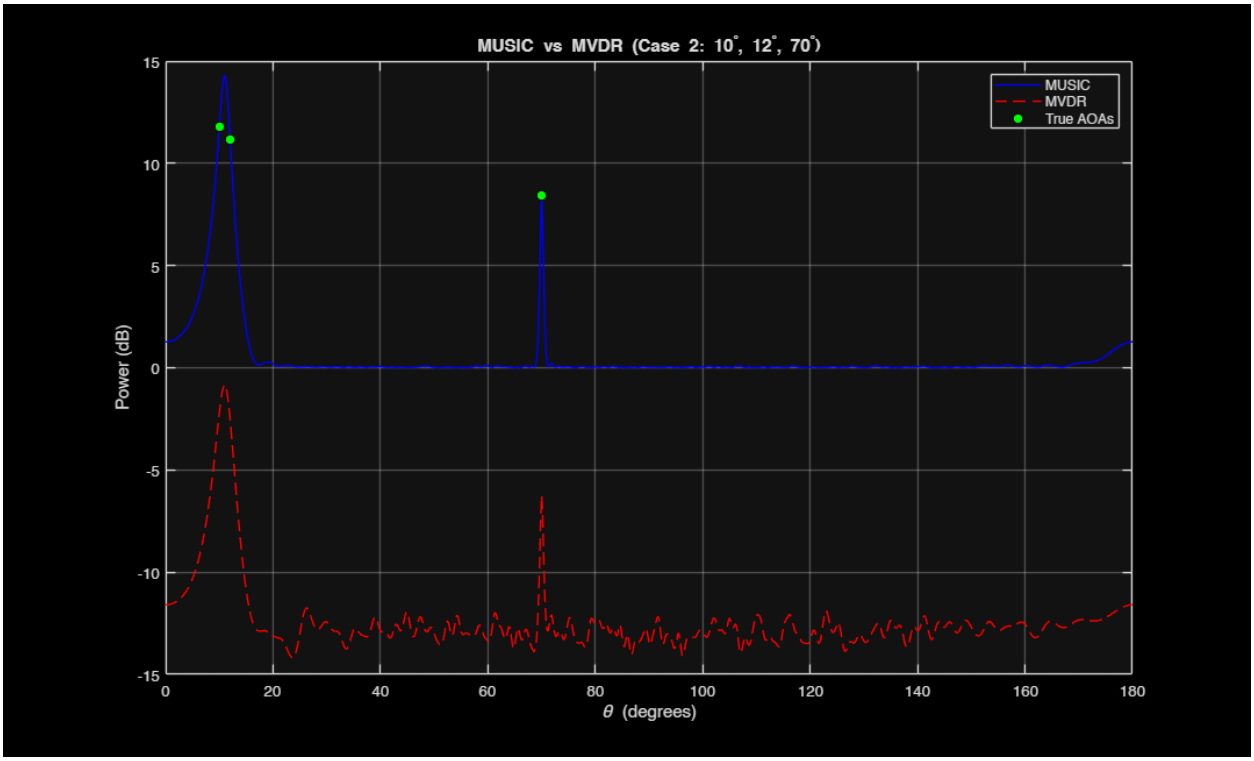
```
    S_MUSIC2(i) = 1 / abs(s_theta' * P2 * s_theta);
    % MVDR: 1 / (s' * R_inv * s)
    S_MVDR2(i) = 1 / abs(s_theta' * Rinverse2 * s_theta);
end


figure;
plot(theta_range, 10*log10(S_MUSIC1), 'b', 'DisplayName', 'MUSIC'); hold on;
plot(theta_range, 10*log10(S_MVDR1), 'r--', 'DisplayName', 'MVDR');

plot(theta1, 10*log10(interp1(theta_range, S_MUSIC1, theta1)), 'ko',
'MarkerFaceColor', 'g', 'DisplayName', 'True AOAs');
title('MUSIC vs MVDR (Case 1: 10^\circ, 25^\circ, 70^\circ)');
xlabel('\theta (degrees)'); ylabel('Power (dB)');
legend; grid on;

figure;
plot(theta_range, 10*log10(S_MUSIC2), 'b', 'DisplayName', 'MUSIC'); hold on;
plot(theta_range, 10*log10(S_MVDR2), 'r--', 'DisplayName', 'MVDR');

plot(theta2, 10*log10(interp1(theta_range, S_MUSIC2, theta2)), 'ko',
'MarkerFaceColor', 'g', 'DisplayName', 'True AOAs');
title('MUSIC vs MVDR (Case 2: 10^\circ, 12^\circ, 70^\circ)');
xlabel('\theta (degrees)'); ylabel('Power (dB)');
legend; grid on;

% compare Eigenvalues of R and singular values of S
for i = 1:3
    fprintf('Eigenvalue \\lambda_%d of R: %.4f | Squared Singular Value \
\sigma_%d^2 of S: %.4f\n', ...
        i, eigval1(i), i, s_vals1(i)^2 / N); % had to divide by N for this
to make sense
end

% check: yes! the relationship holds

for i = 1:3
    alignment = abs(U1(:, i)' * eigvec1(:, i));
    fprintf('Vector %d Alignment (1.0 = perfect multiple): %.4f\n', i,
alignment);
end

% check: yes! the relationship holds

% Case 1
SHS1 = abs(S1' * S1);
disp('Case 1: Absolute values of SHS');
disp(SHS1);

% Case 2
SHS2 = abs(S2' * S2);
disp('Case 2: Absolute values of SHS');
disp(SHS2);
```

```
% these values convey that higher correlated sources (10 deg -> 12 deg in
% case 2) lead to worse noise-signal distinction and the peaks look worse

Sigma Ratio Case 1: 1.4111
Sigma Ratio Case 2: 1.0666
Lambda Ratio Case 1: 1.9913
Lambda Ratio Case 2: 1.1376
Eigenvalue \lambda_1 of R: 1.1105 | Squared Singular Value \sigma_1^2 of S:
1.1105
Eigenvalue \lambda_2 of R: 0.8045 | Squared Singular Value \sigma_2^2 of S:
0.8045
Eigenvalue \lambda_3 of R: 0.5513 | Squared Singular Value \sigma_3^2 of S:
0.5513
Vector 1 Alignment (1.0 = perfect multiple): 1.0000
Vector 2 Alignment (1.0 = perfect multiple): 1.0000
Vector 3 Alignment (1.0 = perfect multiple): 1.0000
Case 1: Absolute values of SHS
    1.0000    0.0190    0.0050
    0.0190    1.0000    0.0081
    0.0050    0.0081    1.0000

Case 2: Absolute values of SHS
    1.0000    0.8273    0.0050
    0.8273    1.0000    0.0068
    0.0050    0.0068    1.0000
```




Singular Values of A (Case 2)
σ
Sorted Eigenvalues of R (Case 2)
λᵢ


MUSIC vs MVDR (Case 1: 10°, 25°, 70°)
MUSIC
MVDR
True AOAs
Power (dB)
θ (degrees)



Published with MATLAB® R2025b